

Double Pointers

What is a Pointer?

A pointer is a container that holds the memory address of another variable, where that variable can be a prototype like an int or double, a structure, or even another pointer.

✓ Think of a pointer as a box; inside the box is a piece of paper with an address written on it. Now think of computer memory as a neighborhood of boxes; each box has an address, just like houses in a neighborhood. But, instead of housing people, each box houses a value. If you declare `unsigned int loop = 0;` you are, in fact, making a new box named “loop” in the memory neighborhood that contains an integer value of 0. And just like a normal house, this box named loop has an address.

If a pointer is its own box, then the pointer has an address that belongs to it, and instead of holding a value like “0” in the box named loop, it holds the address of another box.

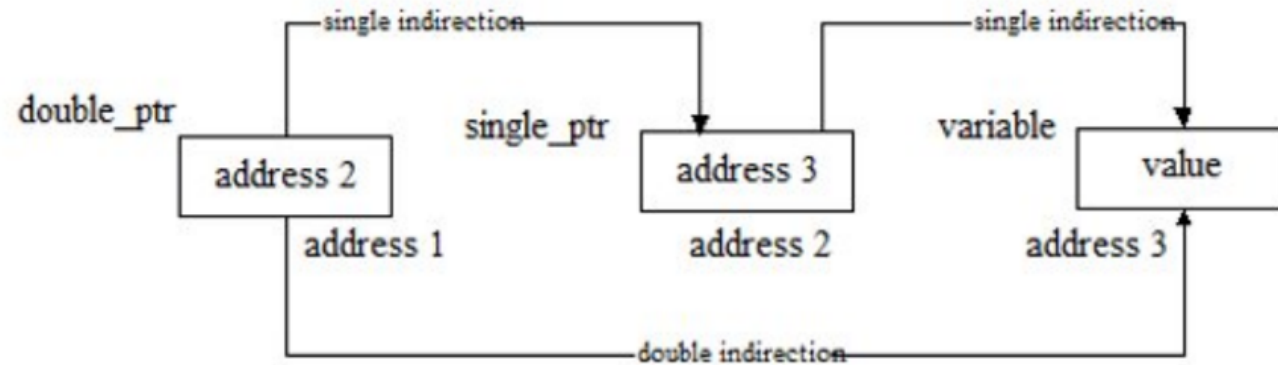
So, if a pointer is a box with an address and holds the address of another box, we can chain them together by having one pointer hold the address of another pointer that, in turn, holds the address of a variable that stores a value.

Why use a double-pointer?

A double pointer is used when we want to be able to update the value of a variable using double indirection. With a double pointer, not only can we update the value in one variable, but we can change the address in the pointer to reference the next variable (usually of the same type) and then update that next variable as well, all without having to change how we are passing that variable to a function.

```
void adminSetup(Org** bikeOrgPtr)
```

This double pointer holds the address of the structure type called Org, which allows us to not only update the values of the variables stored in the structure but also to change the address in the pointer to the address of a different structure. In this way, we chain our structures together, and since a chain is made up of links, we call this a “linked list.”



The diagram above shows how a double pointer works. If we go back to the neighborhood analogy, we could say that we live in the box with address 1, we have a neighbor in address 2, and our friend lives in address 3. Instead of going through our neighbor in address 2 as a middleman, we can interact with our friend in address 3 directly by “asking” for our friend’s address from our neighbor in address 2. Doing this is called “double indirection.”

Structures

```
typedef struct bikerInfo
{
    char bikerName[LENGTH];
    char shirtSize[2];
    char creditCard[16];
    double totalCost;
    double charity;
    struct bikerInfo* nextBikerPtr;
} Biker;
```

A structure is a special type of variable that allows us to group other variables as a whole object. Think of a bicycle and its many parts. The wheels, pedals, frame, and handle all belong to the bike but are distinct pieces and together make up the whole. What makes this special is that just like there are different types of bikes, each with unique pieces, a structure has unique pieces that make it up.

When we want to chain structures in a linked list, we need to add an extra piece to it, a pointer to the same type of structure. This is called a “self-referential” structure because it houses, or “points” to, the address of another structure. The address of the first structure in a list is called the “head pointer.”

When we pass this structure to a function, we do it using a double pointer to the head pointer, so not only can we update the values in this structure, but we can change the address of our pointer using the address stored in the “nextPointer” in the current structure so we can update the values in the next structure in the list.

When we declare a Structure, it is done before the main body of the program. When we do this, we reserve memory on the heap that holds the address for the structure. When we want to allocate memory for this new structure, we need to use the function `malloc()`, which will allocate memory on the heap for the new structure instance. But, when we pass the address of the new structure to a function using a double pointer, the address is passed to the stack because that is where the function is loaded. When we work with the data, it is all on the stack, but when we save that data to the variables within the structure, we work with the memory on the heap because that is where the structure is located.

To work with the data in the structure, we need to be able to read it. This is called “dereferencing.” When we dereference, we ask the variable’s address to tell us what value it is storing. When doing this with structures passed by a double pointer, we use the `&` operator and the arrow operator like this. `¤tPtr->raceCost`. In this case, we are dereferencing the “currentPtr” to get at the value that “raceCost” is storing.

When the program no longer needs the structure, we release the memory allocated to it back to the system using the function `free()`, and then we set the pointer to that structure to NULL to free up that memory on the heap.